

iOS-BLE-SDK 说明文档

修改记录

版本号	修改时间	修改内容
211. 4	2023. 4. 7	初版
211. 6	2023. 5. 10	优化相关接口
211. 8	2023. 6. 6	增加实时数据和血糖
212. 4	2023. 7. 28	增加自定义推送消息

1. 引言

编写目的

对第三方公司提供可以与智能腕表匹配的 iOS 协议 SDK，SDK 中提供主要手环和手机

之间的交互协议。

注:demo 中的蓝牙为基础功能，只做参考，此 sdk 只负责协议部分。

2. API 接入说明

一.将 SDK 的包文件和头文件加入到项目工程中。

二.协议的发送

1.协议的发送参见 STBlueToothSender.h 文件和 STBlueToothString.h 文件。

2.协议说明：协议分为无参数的命令和有参数的命令。无参数的命令传入功能字符串，见 STBlueToothString 文件。有参数的命令具体的见 STBlueToothSender 文件，需要传入响应类型的参数。

三.协议接收。

STBlueToothData 类,负责处理 app 收到的所有数据。返回的数据见文件内说明。主要返回 json 字符串。

3. 接收协议说明

一.蓝牙回调统一采用监听通知 Nof_Revice_Data_Key

例：[[NSNotificationCenter defaultCenter] addObserver:self selector:@selector(nofReviceData:) name:Nof_Revice_Data_Key object:nil];

```
-(void) nofReviceData:(NSNotification *)noti{ if (noti && noti.object) {
```

NSMutableDictionary *dict = noti.userInfo;// dict 即为 SDK 回调返回的命令码数据

NSMutableDictionary *responseObject = noti.object;// responseObject 即为 SDK 回调返回的内容数据

二.dict 格式如下 //{

参数	说明	数据类型
ST_ErrorType_Key	错误码（枚举）	NSNumber
ST_RevType_Key	命令码（枚举）	NSNumber

NSNumber ***errorType** = dict[ST_ErrorType_Key] ; // **revType** 即为 SDK 回调返回的结果枚举

```
typedef enum:NSUInteger {  
    RES_Correct      = 0x00,      /* 命令正确 */  
    RES_CmdError     = 0x01,      /* 命令码错误 */  
    RES_CrcError     = 0x02,      /* 校验码错误 */  
    RES_LenError     = 0x03,      /* 数据长度错误 */  
    RES_Invalid      = 0x04,      /* 数据无效 */  
    RES_Time_Out     = 0x05,      /* 数据超时 */  
}ERROR_TYPE;
```

NSNumber ***revType** = dict[ST_RevType_Key]; // **revType** 即为 SDK 回调返回的命令码枚举

```
typedef enum : NSUInteger {  
    REV_Device_Bind      = 0x81,    //绑定设备  
    REV_Device_State     = 0x02,    //设备状态(获取)  
    SET_Device_State     = 0x82,    //设备状态(设置)  
    REV_Device_Lost      = 0x03,    //查找设备  
    REV_Device_Camera    = 0x04,    //拍照控制  
    REV_Device_Battery   = 0x86,    //获取电池电量  
    REV_Device_Version   = 0x87,    //获取设备版本信息  
    REV_Update_Time      = 0x08,    //时间时区(获取)  
    SET_Update_Time      = 0x88,    //时间时区(设置)  
    REV_User_Info        = 0x09,    //用户信息(获取)  
    SET_User_Info        = 0x89,    //用户信息(设置)  
    REV_Goal_Sport       = 0x0A,    //一天运动目标(获取)  
    SET_Goal_Sport       = 0x8A,    //一天运动目标(设置)  
    REV_Device_Pair      = 0x8B,    //蓝牙系统配对  
    REV_State_Notification = 0x0C,    //消息推送开关(获取)  
    SET_State_Notification = 0x8C,    //消息推送开关(设置)  
    REV_Health_Current   = 0x8D,    //获取当前设备运动数据  
    REV_Health_Switch     = 0x0E,    //健康数据检测开关(获取)  
    SET_Health_Switch     = 0x8E,    //健康数据检测开关(设置)  
    SET_Factory_Setting   = 0x8F,    //恢复出厂设置  
  
    REV_Interval_HR      = 0x31,    //心率检测间隔(获取)  
    SET_Interval_HR      = 0xB1,    //心率检测间隔(设置)  
    REV_Contact_Common   = 0x32,    //常用联系人(获取)  
    SET_Contact_Common   = 0xB2,    //常用联系人(设置)  
    REV_Contact_SOS      = 0x33,    //紧急联系人(获取)  
    SET_Contact_SOS      = 0xB3,    //紧急联系人(设置)  
    REV_Remind_NoDisturb  = 0x34,    //勿扰(获取)  
    SET_Remind_NoDisturb  = 0xB4,    //勿扰(设置)
```

```

REV_Remind_AlarmClock = 0x35, //闹钟(获取)
SET_Remind_AlarmClock = 0xB5, //闹钟(设置)
REV_Remind_Sedentary = 0x36, //久坐提醒(获取)
SET_Remind_Sedentary = 0xB6, //久坐提醒(设置)
REV_Remind_DrinkWater = 0x37, //喝水提醒(获取)
SET_Remind_DrinkWater = 0xB7, //喝水提醒(设置)
SET_Info_Weather = 0xB9, //推送天气
REV_Alarm_Even = 0x3B, //事件提醒(获取)
SET_Alarm_Even = 0xBB, //事件提醒(设置)

REV_History_Sport = 0xE1, //同步运动
REV_History_Step = 0xE2, //同步记步/睡眠
REV_History_HR = 0xE3, //同步心率
REV_History_BP = 0xE4, //同步血压
REV_History_BQ = 0xE5, //同步血氧
REV_History_Pressure = 0xE6, //同步压力
REV_History_Met = 0xE7, //同步梅脱
REV_History_Temp = 0xE8, //同步温度
REV_History_ValidDate = 0xE9, //获取有效日期列表

REV_Bin_File_Info = 0xEA, //发送文件信息
REV_Bin_File = 0xEB, //文件传输

REV_Dial_Info = 0xEC, //获取表盘信息
SET_Dial_Current = 0xED, //切换当前显示表盘
REV_History_Mai = 0xEE, //同步Mai

ET_RealTime_Switch = 0x70, //实时数据开关(设置)
REV_RealTime_Switch = 0xF1, //实时数据开关(获取)
REV_RealTime_DATA = 0x71, //实时数据上发(获取)
REV_ShutDown_CMD = 0x90, //设备关机

REV_History_Sugar = 0xF2, //同步血糖

}REV_TYPE;

```

三. responseObject 格式如下 //{

1. 收到绑定

参数	说明	数据类型
ST_DeviceBindKey	绑定是否成功(1:成功 0:失败)	NSNumber

2. 设备状态

参数	说明	数据类型
ST_HourSystemKey	时间格式设置 (0: 24 小时制, 1: 12 小时制)	NSNumber
ST_UnitKey	单位制选择 (0: 公制, 1: 英制)	NSNumber
ST_TemperatureUnitKey	温度制选择 (0: 摄氏, 1: 华氏)	NSNumber
ST_LanguageDataKey	手表语言 (0 中文简, 1 中文繁, 2 英文, 3 俄语, 4 法语, 5 西班牙语, 6 德语, 7 日语, 8 意大利语, 9 韩语, 10, 荷兰语, 11 泰语)	NSNumber
ST_BrightDurationKey	背光时长 (秒为单位)	NSNumber
ST_BrightnessKey	屏幕亮度 (百分比)	NSNumber
ST_TruncWristKey	抬手亮开关	NSNumber

3. 查找设备

参数	说明	数据类型
ST_FindDeviceKey	查看手机 (1:成功 0:失败)	NSNumber

4. 拍照控制

参数	说明	数据类型
ST_PhotoGraphModeKey	1: (进入拍照界面) 2: (退出拍照界面) 3: (app 点击拍照)	NSNumber

5. 获取电池电量

参数	说明	数据类型
ST_GetElectricityStateKey	获取充电状态 (1:充电中)	NSNumber
ST_GetElectricityKey	电量百分比	NSNumber

6. 获取设备信息

参数	说明	数据类型
----	----	------

ST_GetVersionNumberKey	固件版本号	NSString
ST_GetUIVersionNumberKey	固件 UI 版本号	NSString
ST_GetDeviceMtuKey	设备接收 buf 大小	NSNumber
ST_GetDeviceWidthKey	设备 lcd 宽	NSNumber
ST_GetDeviceHeightKey	设备 lcd 高	NSNumber
ST_GetDeviceShapeKey	获取设备形状 (0: 圆屏, 1: 方屏)	NSNumber
ST_GetDeviceModelKey	设备型号批次号 (字符串 X01G001)	NSString
ST_ForcedUpgradeUIKey	是否需要强制升级 UI	NSNumber

7. 获取时间时区

参数	说明	数据类型
ST_YearKey	年	NSNumber
ST_MonthKey	月	NSNumber
ST_DayKey	日	NSNumber
ST_HourKey	时	NSNumber
ST_MinuteKey	分	NSNumber
ST_SecondKey	秒	NSNumber
ST_ZoneKey	时区	NSNumber

8. 用户信息

参数	说明	数据类型
ST_UserGenderKey	性别, 0: 女, 1: 男	NSNumber
ST_UserAgeKey	年龄	NSNumber
ST_UserHeightKey	身高 (单位 1 CM)	NSNumber
ST_UserWeightKey	体重 (单位 0.1 KG)	NSNumber

9. 设置一天运动目标

参数	说明	数据类型
ST_TargetStepKey	计步目标	NSNumber
ST_TargetCalorieKey	热量目标 (千卡)	NSNumber
ST_TargetDistanceKey	距离目标 (千米)	NSNumber

10. 消息推送开关

参数	说明	数据类型
ST_NotiAllKey	总开关: 1 开 0 关	NSNumber
ST_NotiIncomingKey	来电开关: 1 开 0 关	NSNumber
ST_NotiSmsKey	短信开关: 1 开 0 关	NSNumber
ST_NotiEmailKey	mail 开关: 1 开 0 关	NSNumber
ST_NotiTwitterKey	Twitter 开关: 1 开 0 关	NSNumber
ST_NotiFacebookKey	Facebook 开关: 1 开 0 关	NSNumber
ST_NotiWhatsappKey	WhatsApp 开关: 1 开 0 关	NSNumber
ST_NotiLineKey	Line 开关: 1 开 0 关	NSNumber
ST_NotiSkypeKey	Skype 开关: 1 开 0 关	NSNumber
ST_NotiQqKey	QQ 开关: 1 开 0 关	NSNumber
ST_NotiWechatKey	wechat 开关: 1 开 0 关	NSNumber
ST_NotiInstagramKey	Instagram 开关: 1 开 0 关	NSNumber
ST_NotiLinkedinKey	LinkedIn 开关: 1 开 0 关	NSNumber
ST_NotiMessengerKey	Messenger 开关: 1 开 0 关	NSNumber
ST_NotiVkKey	VK 开关: 1 开 0 关	NSNumber
ST_NotiViberKey	Viber 开关: 1 开 0 关	NSNumber
ST_NotiTelegramKey	Telegram 开关: 1 开 0 关	NSNumber
ST_NotiKakaoTalkKey	KakaoTalk 开关: 1 开 0 关	NSNumber
ST_NotiOtherKey	其他开关: 1 开 0 关	NSNumber

11. 获取当前设备展示数据

参数	说明	数据类型
ST_GetCurrentValueStepKey	总计步	NSNumber
ST_GetCurrentValueCalorieKey	总卡路里(卡)	NSNumber
ST_GetCurrentValueDistanceKey	总距离(m)	NSNumber
ST_GetCurrentValueSleepKey	总睡眠时间(分钟)	NSNumber
ST_GetCurrentValueDeepSleepKey	总深睡时间(分钟)	NSNumber
ST_GetCurrentValueLightSleepKey	总浅睡时间(分钟)	NSNumber
ST_GetCurrentValueHRKey	心率	NSNumber
ST_GetCurrentValueHeightBPKey	高压	NSNumber
ST_GetCurrentValueLowBPKey	低压	NSNumber
ST_GetCurrentValueBOKey	血氧	NSNumber
ST_GetCurrentValueHPKey	压力	NSNumber
ST_GetCurrentValueMETKey	梅脱	NSNumber
ST_GetCurrentValueMAIKey	MAI	NSNumber
ST_GetCurrentValueTemperatureKey	温度 (显示需除以 100)	NSNumber

12. 获取健康数据检测开关

参数	说明	数据类型
ST_SwitchHRKey	心率总开关: 1 开 0 关	NSNumber
ST_SwitchBPKey	血压总开关: 1 开 0 关	NSNumber
ST_SwitchBOKey	血氧总开关: 1 开 0 关	NSNumber
ST_SwitchPPKey	压力总开关: 1 开 0 关	NSNumber
ST_SwitchTPKey	温度总开关: 1 开 0 关	NSNumber

13. 心率检测间隔和范围

参数	说明	数据类型
ST_StartHourKey	开始小时	NSNumber
ST_StartMinsKey	开始分钟	NSNumber
ST_EndHourKey	结束小时	NSNumber
ST_EndMinsKey	结束分钟	NSNumber
ST_IntervalKey	检测周期 (以分钟为单位)	NSNumber
ST_ThresholdKey	静态心率高报警阈值	NSNumber

14. 常用/紧急联系人

参数	说明	数据类型
ST_ContactsKey	联系人	NSArray
ST_ContactNameKey	姓名	NSString
ST_ContactPhoneKey	电话	NSString

15. 勿扰

参数	说明	数据类型
ST_StartHourKey	开始小时	NSNumber
ST_StartMinsKey	开始分钟	NSNumber
ST_EndHourKey	结束小时	NSNumber
ST_EndMinsKey	结束分钟	NSNumber
ST_NoDisturbOpenKey	定时开关	NSNumber
ST_NoDisturbAllDayKey	是否全天勿扰	NSNumber

16. 闹钟

参数	说明	数据类型
ST_ClocksKey	闹钟	NSArray
ST_ClockHourKey	时	NSNumber
ST_ClockMinsKey	分	NSNumber
ST_ClockOpenKey	开关	NSNumber
ST_ClockCycleKey	星期重复(6-0)	NSString
ST_ClockTypeKey	类型	NSNumber

17. 久坐/喝水提醒

参数	说明	数据类型
ST_StartHourKey	开始小时	NSNumber
ST_StartMinsKey	开始分钟	NSNumber
ST_EndHourKey	结束小时	NSNumber
ST_EndMinsKey	结束分钟	NSNumber
ST_AlarmOpenKey	开关	NSNumber
ST_AlarmIntervalKey	提醒间隔(单位分钟)	NSNumber

18. 同步运动数据

参数	说明	数据类型
ST_YearKey	年	NSNumber
ST_MonthKey	月	NSNumber
ST_DayKey	日	NSNumber
ST_HourKey	时	NSNumber
ST_MinuteKey	分	NSNumber
ST_SecondKey	秒	NSNumber
ST_GetSportRecordDurationKey	运动时长(s)	NSNumber
ST_GetSportRecordTypeKey	运动类型	NSNumber
ST_GetSportRecordStepKey	步数	NSNumber
ST_GetSportRecordDistanceKey	距离(m)	NSNumber
ST_GetSportRecordSpeedKey	速度(m/s)	NSString
ST_GetSportRecordCaloriesKey	卡路里	NSNumber
ST_GetSportRecordPaceKey	配速	NSNumber
ST_GetSportRecordStrideKey	步频	NSNumber
ST_GetSportRecordHRLengthKey	心率长度	NSNumber

ST_GetSportRecordHRDetailsKey	心率数据	NSString
-------------------------------	------	----------

19. 同步计步/卡路里/距离/睡眠数据

参数	说明	数据类型
ST_GetRecordDateTimeKey	年月日 (yyyyMMdd)	NSString
ST_GetRecordIntervalKey	历史采样间隔 (mins)	NSNumber
ST_GetRecordValueDataKey	历史数据	NSArray
ST_GetRecordTypeStepOrSleepKey	数据类型 (1 计步、2 睡眠)	NSNumber
ST_GetRecordValueStepKey	计步	NSNumber
ST_GetRecordValueCalorieKey	卡路里 (卡)	NSNumber
ST_GetRecordValueDistanceKey	距离 (m)	NSNumber
ST_GetRecordValueSleepKey	睡眠 (1 开始入睡 2 浅睡 3 深睡 4 清醒)	NSNumber

20. 同步心率/血压/血氧/压力/梅脱/温度/MAL/血糖数据

参数	说明	数据类型
ST_GetRecordDateTimeKey	年月日 (yyyyMMdd)	NSString
ST_GetRecordIntervalKey	历史采样间隔 (mins)	NSNumber
ST_GetRecordValueDataKey	历史数据: 1 分钟一组 (高低压 2 个为 1 组, 心率/血压/血氧/压力为 0 和 255 为异常数据, 需过滤)	NSString

21. 获取数据有效日期列表

参数	说明	数据类型
ST_GetRecordValidDateKey	有效日期列表	NSArray
ST_YearKey	年	NSNumber
ST_MonthKey	月	NSNumber
ST_DayKey	日	NSNumber

22. 获取表盘信息

参数	说明	数据类型
ST_GetDialInfoKey	表盘信息列表	NSArray
ST_GetDialUsedKey	为 1 表示当前显示表盘	NSNumber
ST_GetDialIdKey	表盘 id	NSNumber

ST_GetDialRGBKey	颜色值 (rgb565)	NSNumber
ST_GetDialModeKey	位置 1: 5 上 2: 中 3: 下	NSNumber

23. 获取事件提醒信息

参数	说明	数据类型
ST_EvensKey	事件提醒列表	NSArray
ST_EvenYearKey	年	NSNumber
ST_EvenMonthKey	月	NSNumber
ST_EvenDayKey	日	NSNumber
ST_ClockHourKey	时	NSNumber
ST_ClockMinsKey	分	NSNumber
ST_EventTypeKey	提醒类型(1、2、3、4)	NSNumber
ST_EventCycleTypeKey	重复类型 (1、单次 2: 天 3: 周 4: 月 5: 年)	NSNumber
ST_EventCycleValueKey	周重复值 (bit0 周一)	NSString
ST_EventContentKey	提醒内容	NSString

24. 实时数据开关

参数	说明	数据类型
ST_GetRealTimeGsensorOnKey	gsensor	NSNumber
ST_GetRealTimeSdcOnKey	计步、卡路里、距离	NSNumber
ST_GetRealTimeHrOnKey	心率	NSNumber
ST_GetRealTimeBpOnKey	血压	NSNumber
ST_GetRealTimeBqOnKey	血氧	NSNumber
ST_GetRealTimeTpOnKey	温度	NSNumber
ST_GetRealTimeBsOnKey	血糖	NSNumber

4. 发送协议说明

一. STBlueToothSender 类

例:

第一步: NSData *data = [STBlueToothSender writeDeviceDateTime];

第二步: [STBleManager.sharedInstance writeCommand:data];

以下解析的是“第一步”发送数据的详情

1. 绑定设备

```
+ (NSData *)writeDeviceBind;
```

2. 设备状态

```
+ (NSData *)readDeviceState;  
+ (NSData *)writeDeviceState:(STDeviceState *)model;
```

STDeviceState 详情

参数	说明	数据类型
STTimeMode	时间制式	enum
STUnit	单位	enum
STTemperatureUnit	温度单位	enum
STLanguage	语言选择	enum
brightDuration	亮屏时长(秒为单位)	NSInteger
brightness	屏幕亮度(百分比)	NSInteger
trunWrist	翻腕亮屏(开关)	BOOL

3. 查找设备

```
+ (NSData *)writeFindDevice;
```

4. 拍照控制

```
+ (NSData *)writePhotoContrl:(STPhotoContrl)contrl;
```

STPhotoContrl 详情

```
typedef enum : NSUInteger {  
    /// 进入拍照  
    STPhotoContrlStart = 0x01,  
    /// 退出拍照  
    STPhotoContrlEnd = 0x02,  
    /// 点击拍照  
    STPhotoContrlTakePhoto = 0x03,  
  
}STPhotoContrl;
```

5. 获取电池电量

```
+ (NSData *)readDeviceBattery;
```

6. 获取设备版本信息

+ (NSData *)readDeviceVersion;

7. 时间时区

+ (NSData *)readDeviceDateTime;

+ (NSData *)writeDeviceDateTime;

8. 用户信息

+ (NSData *)readUserInfo;

+ (NSData *)writeUserInfo:(STUserInfo *)model;

STUserInfo 详情

参数	说明	数据类型
sex	性别(默认 0: 女, 1: 男)	NSInteger
age	年龄 (0x06~0x64, 默认 20)	NSInteger
height	身高 (单位 cm, 默认 170)	NSInteger
weight	体重(单位 0.1kg, 默认 600)	NSInteger

9. 一天运动目标

+ (NSData *)readSportGoal;

+ (NSData *)writeSportGoal:(STISportGoal *)model;

STISportGoal 详情

参数	说明	数据类型
step	计步目标	NSInteger
calories	卡路里目标(单位大卡)	NSInteger
distance	距离目标(单位 KM)	NSInteger

10. 蓝牙系统配对

+ (NSData *)writeDevicePair;

11. 消息推送开关

+ (NSData *)readMessageNotice;

+ (NSData *)writeMessageNotice:(STMessageNotice *)model;

STMessageNotice 详情

参数	说明	数据类型
----	----	------

isAll	总开关: 1 开 0 关	BOOL
isIncoming	来电开关: 1 开 0 关	BOOL
isSms	短信开关: 1 开 0 关	BOOL
isEmail	mail 开关: 1 开 0 关	BOOL
isTwitter	Twitter 开关: 1 开 0 关	BOOL
isFacebook	Facebook 开关: 1 开 0 关	BOOL
isWhatsapp	WhatsApp 开关: 1 开 0 关	BOOL
isLine	Line 开关: 1 开 0 关	BOOL
isSkype	Skype 开关: 1 开 0 关	BOOL
isQq	QQ 开关: 1 开 0 关	BOOL
isWechat	wechat 开关: 1 开 0 关	BOOL
isInstagram	Instagram 开关: 1 开 0 关	BOOL
isLinkedin	LinkedIn 开关: 1 开 0 关	BOOL
isMessenger	Messenger 开关: 1 开 0 关	BOOL
isVk	VK 开关: 1 开 0 关	BOOL
isViber	Viber 开关: 1 开 0 关	BOOL
isTelegram	Telegram 开关: 1 开 0 关	BOOL
isKakaoTalk	KakaoTalk 开关: 1 开 0 关	BOOL
isOther	Other 开关: 1 开 0 关	BOOL

12. 获取当前设备展示数据

```
+ (NSData *)readCurrentHealth;
```

13. 健康数据检测开关

```
+ (NSData *)readHealthSwitch;
```

```
+ (NSData *)writeHealthSwitch: (STHealthSwitch *)model;
```

STHealthSwitch 详情

参数	说明	数据类型
isHR	心率总开关: 1 开 0 关	BOOL
isBP	血压总开关: 1 开 0 关	BOOL
isBO	血氧总开关: 1 开 0 关	BOOL
isPP	压力总开关: 1 开 0 关	BOOL
isTP	温度总开关: 1 开 0 关	BOOL
isBS	血糖总开关: 1 开 0 关	BOOL

14. 恢复出厂设置

```
+ (NSData *)writeFactorySetting;
```

#pragma mark - 复杂设置类

16. 心率检测间隔和范围

```
+ (NSData *)readConfigMeasureHR;  
+ (NSData *)writeConfigMeasureHR:(STConfigMeasureHR *)model;
```

STConfigMeasureHR 详情

参数	说明	数据类型
STIntervalTime	时间段	NSObject
interval	检测周期(以分钟为单位)	NSInteger
thresholdHR	静态心率高报警阈值	NSInteger

STIntervalTime 详情

参数	说明	数据类型
startHour	开始小时	NSInteger
startMinsv	开始分钟	NSInteger
endHour	结束小时	NSInteger
endMins	结束分钟	NSInteger

17. 常用联系人

```
+ (NSData *)readCommonContacts;  
+ (NSData *)writeCommonContacts:(NSArray<STContacts *> *)modelArr;
```

STContacts 详情

参数	说明	数据类型
isSOS	是否紧急联系人	BOOL
name	姓名	NSString
phone	电话	NSString

18. 紧急联系人

```
+ (NSData *)readSosContacts;  
+ (NSData *)writeSosContacts:(NSArray<STContacts *> *)modelArr;
```

19. 勿扰

```
+ (NSData *)readNoDisturb;
+ (NSData *)writeNoDisturb:(STNoDisturb *)model;
```

STNoDisturb 详情

参数	说明	数据类型
isOpen	是否定时勿扰	BOOL
isAllDay	是否全天勿扰	BOOL
STIntervalTime	时间段	NSObject

20. 闹钟

```
+ (NSData *)readAlarmClocks;
+ (NSData *)writeAlarmClocks:(NSArray<STAlarmClock *> *)modelArr;
```

STAlarmClock 详情

参数	说明	数据类型
isOpen	闹钟开关	BOOL
hour	小时	NSInteger
minute	分钟	NSInteger
STAlarmClockType	类型	enum
cycle	周期(每天默认都是 false+(6-0)+(每周日-每周六))	NSArray

21. 久坐提醒

```
+ (NSData *)readSedentaryAlarmInterval;
+ (NSData *)writeSedentaryAlarmInterval:(STAlarmInterval *)model;
```

STAlarmInterval 详情

参数	说明	数据类型
isOpen	开关	BOOL
STIntervalTime	时间段	NSObject
interval	提醒间隔(单位分钟)	NSInteger

22. 喝水提醒

```
+ (NSData *)readDrinkWaterAlarmInterval;
+ (NSData *)writeDrinkWaterAlarmInterval:(STAlarmInterval *)model;
```


23. 推送天气

```
+ (NSData *)writeWeather:(NSArray<STWeather *> *)modelArr;
```

STWeather 详情

参数	说明	数据类型
temp	温度	NSString
tempMin	最低温度	NSString
tempMax	最高温度	NSString
conditionCode	天气状态代号 (1、小雨, 2、中雨, 3、大雨, 4、阴天, 5、多云, 6、晴, 7、雾霾, 8、台风, 9、雷雨, 10、冰雹 11、小雪, 12、中雪, 13、大雪, 14、雨夹雪15、沙尘暴, 16、学加冰雹, 17、狂风, 18、大风, 19、小风, 20、龙卷风, 21、热带风暴, 22, 雷暴, 23, 猛烈雷暴, 24、未知)	NSString
unit	单位, 0 摄氏, 1 华氏	BOOL
windSpeed	风速	NSString
humidity	湿度 (相对湿度, 百分比数值	NSString
vis	能见度, 默认单位: 公里	NSString
uvIndex	紫外线强度	NSString
AQI	空气质量 (1 优, 2 良, 3 差)	NSString

24. 事件提醒

```
+ (NSData *)readAlarmEvents;
```

#pragma mark - 数据同步类

25. 同步运动数据 (是否重发 1: 重发, 0: 不重发)

```
+ (NSData *)readSportModeHistory:(BOOL)resend;
```

26. 同步计步/卡路里/距离/睡眠数据

```
///
```

```
/// 获取某一天的历史记录数据
```

```
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readStepAndSleepHistoryWithDate:(NSString * _Nonnull)dateStr;
```

27. 同步心率数据

```
///
/// 获取某一天的历史记录数据
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readHeartRateHistoryWithDate:(NSString * _Nonnull)dateStr;
```

28. 同步血压数据

```
///
/// 获取某一天的历史记录数据
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readBloodPressureHistoryWithDate:(NSString * _Nonnull)dateStr;
```

29. 同步血氧数据

```
///
/// 获取某一天的历史记录数据
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readBloodOxygenHistoryWithDate:(NSString * _Nonnull)dateStr;
```

30. 同步压力数据

```
///
/// 获取某一天的历史记录数据
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readPhysicalPressureHistoryWithDate:(NSString * _Nonnull)dateStr;
```

31. 同步梅脱数据

```
///
/// 获取某一天的历史记录数据
/// \param dateStr 日期yyyyMMdd
///
+ (NSData *)readMetsHistoryWithDate:(NSString * _Nonnull)dateStr;
```

32. 同步温度数据

```
///  
/// 获取某一天的历史记录数据  
/// \param dateStr 日期yyyyMMdd  
///  
+ (NSData *)readTemperatureHistoryWithDate:(NSString * _Nonnull)dateStr;
```

33. 同步Mai数据

```
///  
/// 获取某一天的历史记录数据  
/// \param dateStr 日期yyyyMMdd  
///  
+ (NSData *)readMaiHistoryWithDate:(NSString * _Nonnull)dateStr;
```

34. 同步血糖数据

```
///  
/// 获取某一天的历史记录数据  
/// \param dateStr 日期yyyyMMdd  
///  
+ (NSData *)readSugarHistoryWithDate:(NSString * _Nonnull)dateStr;
```

35. 获取数据有效日期列表(对应数据类型)

```
+ (NSData *)readHistoryValidDate:(STHistoryCmd)cmd;
```

STHistoryCmd 详情

```
typedef enum : NSUInteger {  
    ST_History_Sport      = 0x61,    //同步运动  
    ST_History_Step       = 0x62,    //同步记步/睡眠  
    ST_History_HR         = 0x63,    //同步心率  
    ST_History_BP         = 0x64,    //同步血压  
    ST_History_BQ         = 0x65,    //同步血氧  
    ST_History_Pressure   = 0x66,    //同步压力  
    ST_History_Met        = 0x67,    //同步梅脱  
    ST_History_Temp       = 0x68,    //同步温度  
    ST_History_Mai        = 0x6E,    //同步Mai  
}
```

```
}STHistoryCmd;
```

#pragma mark - 表盘设置类

36. 获取表盘信息

```
+ (NSData *)readDeviceDialInfo;
```

37. 切换当前显示表盘

```
+ (NSData *)writeCurrentDeviceDial:(int)Id;
```

备注, Id: 手表表盘ID

#pragma mark - UIImage转RGB565

```
+ (NSData *)getRGB565DataWith:(UIImage*)image;
```

//RGB888转RGB565数组

```
+ (NSData *)getRGB565DataWithR:(unsigned char)r G:(unsigned char)g B:(unsigned char)b;
```

#pragma mark - 实时上报

38. 实时数据开关(获取)

```
+ (NSData *)readRealTimeSwitchs;
```

39. 实时数据开关(设置)

```
+ (NSData *)writeRealTimeSwitchs:(STRealTimeSwitchs *)model;
```

STRealTimeSwitchs 详情

参数	说明	数据类型
gsensor	gsensor 实时数据	BOOL
sdc	计步、卡路里、距离实时数据	BOOL
hr	心率实时数据	BOOL
bp	血压实时数据	BOOL
bq	血氧实时数据	BOOL
tp	温度实时数据	BOOL
bs	血糖实时数据	BOOL

40. 设置关机(设置)

```
+ (NSData *)writeShutDown;
```

二. STBlueToothData 类

例:

```
第一步: STBlueToothData *sender = [STBlueToothData sharedInstance];
```

41. 写入bin文件数据

```
- (void)writeBinFileInfo:(nonnull NSData *)info
    Data:(nonnull NSData *)data
    BlockReadInterval:(NSUInteger)blockReadInterval
    Peripheral:(nonnull CBPeripheral *)peripheral
    Characteristic:(nonnull CBCharacteristic *)characteristic
    Progress:(ProgressBlock)progress;
```

参数详情

参数	说明	数据类型
info	文件简要	NSData
data	文件内容	NSData
blockReadInterval	设备最大写入长度	NSUInteger
peripheral	设备对象	CBPeripheral
characteristic	设备写特征值	CBCharacteristic
progress	写入的进度结果	ProgressBlock

```
//事件提醒(设置)
- (void)writeAlarmEvents:(NSArray<STAlarmEvent *> *)modelArr
    BlockReadInterval:(NSUInteger)blockReadInterval
    Peripheral:(nonnull CBPeripheral*)peripheral
    Characteristic:(nonnull CBCharacteristic*)characteristic;
```

STAlarmEvent 详情

参数	说明	数据类型
year	年	NSInteger
month	月	NSInteger

day	日	NSInteger
hour	时	NSInteger
minute	分	NSInteger
STAlarmEventType	类型	enum
contentLen	提醒内容长度	NSString
content	提醒内容（Unicode 编码，n 最大 254 字节-127 个字符）	NSString
cycleType	重复类型（1、单次 2：天 3，：周 4：月 5：年）	NSInteger
cycle	周重复值（bit0 周一）	NSArray

42. 自定义推送消息(设置)

```
+ (NSData *)writePushMessageTitle:(NSString *)title Message:(NSString *)message;
```

参数详情

参数	说明	数据类型
title	标题	NSString
message	内容	NSString

43. 同步新睡眠数据

```
///  
///获取某一天的历史记录数据  
///\paramdateStr日期yyyyMMdd  
///  
+(NSData*)readNewSleepHistoryWithDate:(NSString*_Nonnull)dateStr;
```

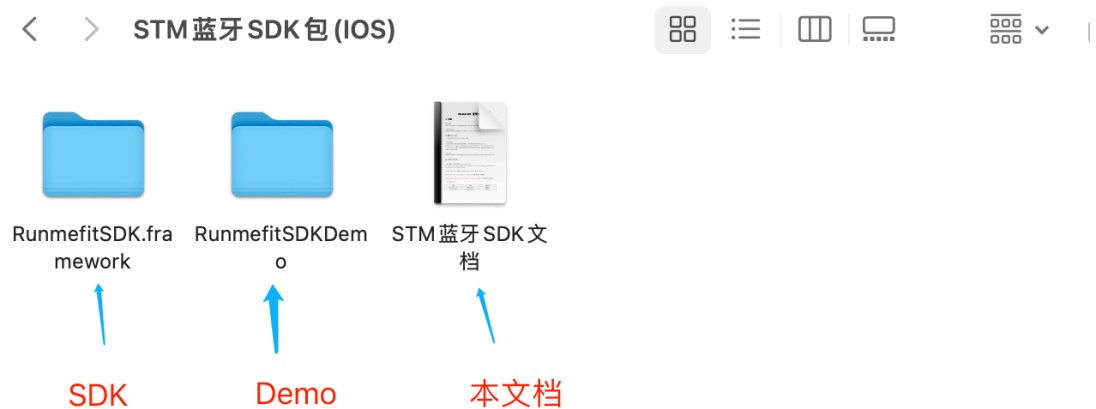
5. SDK 使用原则

注：开发者调用 SDK 实现业务逻辑时，请严格按照 [蓝牙准备好]->[搜索设备]->[连接设备]->[设备准备好读写数据]->[功能接口] 的顺序.

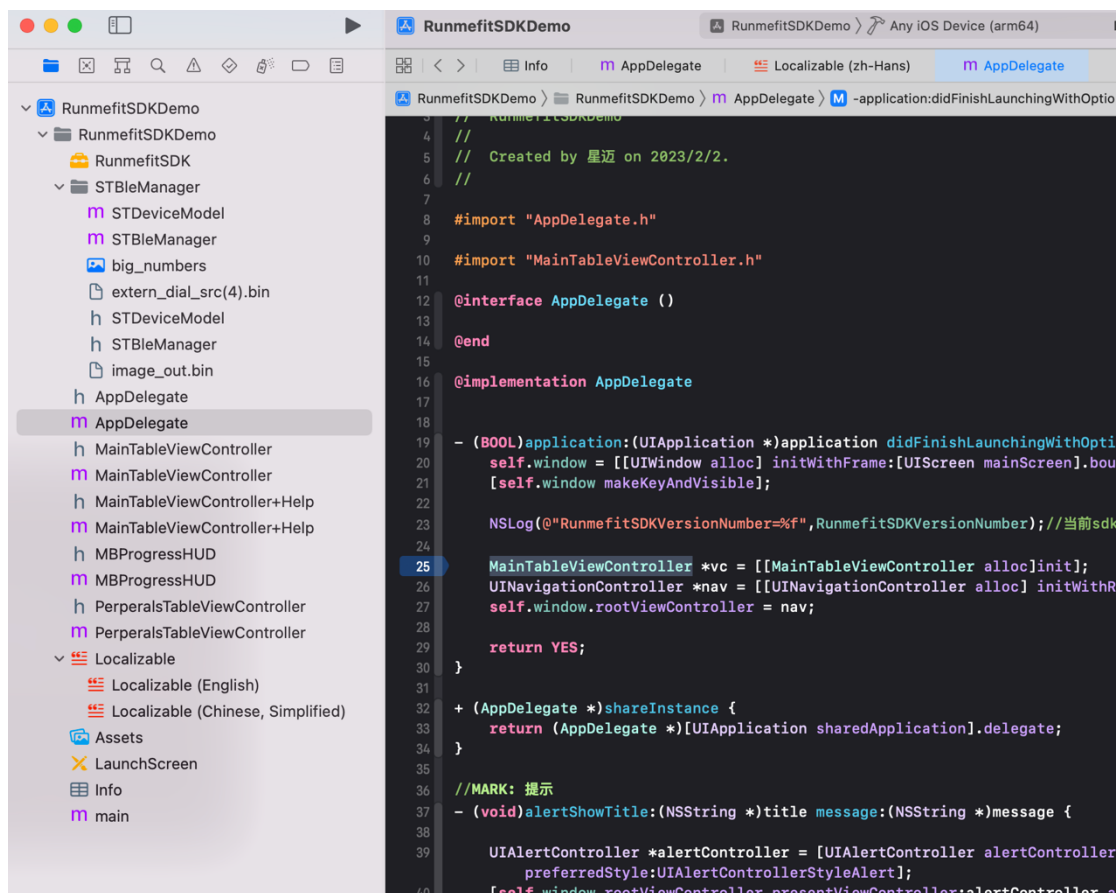
一. 使用范围

iOS-SDK 适用于 BLE 4.0 , iOS 8.0 及以上版本 xcode 版本 10.0 以上
支持语言 Objective-c、Swift4.0 及以上

二. 手动集成 xcode 集成



手动添加 RunmefitSDK.framework 至您的工程目录下。如图所示



三. 设备相关操作

1. 初始化

蓝牙连接相关的类为单例，需要用的时候直接调用以下方法获取对象
STBLEManager.sharedInstance

2. 开始准备搜索

a. 搜索设备前, 请务必确认蓝牙处于 power on 状态, 请使用 STBLEManager.sharedInstance setUpdateState 回调方法来确定蓝牙状态

```
@property(nonatomic, assign) BOOL stateOn;///系统蓝牙开关

@property (nonatomic, strong) NSMutableArray<STDeviceModel*> *deviceModels; ///搜索设备
```

开始搜索: STBLEManager.sharedInstance startScan];

停止搜索: [STBLEManager.sharedInstance stopScan];

b. 回调方法来确定搜索蓝牙设备

```
[STBLEManager.sharedInstance setUpdatePeripheral:^(NSArray<STDeviceModel *> *
_Nonnull deviceModels) {

}];
```

STDeviceModel 详情

参数	说明	数据类型
peripheral	设备对象	CBPeripheral
name	设备名称	NSString
mac	设备 Mac 地址	NSString
rssI	设备信号	NSNumber

3. 连接设备

a. 将蓝牙设备模型传入连接方法

```
[STBLEManager.sharedInstance connectPeripheral:deviceModel];
```

b. 回调方法来确定蓝牙连接状态

```
[STBLEManager.sharedInstance setUpdateConnect:^(BOOL connect) {
    if (connect == YES) {
        NSLog(@"%@", TRLocalizedString(@"连接成功", nil));
    }else{
        NSLog(@"%@", TRLocalizedString(@"连接失败", nil));
    }
}
```

```
    }  
  }];
```

4. 断开设备

a. 将蓝牙设备模型传入连接方法

```
[STBLEManager.sharedInstance cancelPeripheral:deviceModel];
```

b. 回调方法来确定蓝牙连接状态

```
[STBLEManager.sharedInstance setUpdateConnect:^(BOOL connect) {  
    if (connect == YES) {  
        NSLog(@"%@", TRLocalizedString(@"连接成功", nil));  
    } else {  
        NSLog(@"%@", TRLocalizedString(@"连接断开", nil));  
    }  
}];
```

5. 发送指令

```
[STBLEManager.sharedInstance writeCommand:data];
```